

Competitive Programming

Section 39: Heaps and Priority Queues

Competitive Programming Course

2026-07-06

Table of contents

1	Priority queues as frontier managers	3
2	Two heaps for medians	4
3	Sweep with ending times	5
4	K-way merging	6
5	Python and C++ heap details	7
6	Checklist	8
7	Local exercises	9
8	How to use the checker	10

1 Priority queues as frontier managers

A heap maintains the smallest or largest currently available item. It is the right tool when the next action is determined by one key, but future insertions can change the frontier.

You should be able to say what one heap entry means. Examples:

<code>(value)</code>	<code>current candidate number</code>
<code>(end_time)</code>	<code>room that becomes free at this time</code>
<code>(sum, i, j)</code>	<code>pair $A[i]+B[j]$ from stream i at position j</code>
<code>(distance, vertex)</code>	<code>frontier state in Dijkstra</code>

The heap does not prove the algorithm by itself. The proof comes from showing that the smallest or largest key is safe to process next.

2 Two heaps for medians

Keep the lower half in a max-heap and the upper half in a min-heap:

```
max(low) <= min(high)
len(low) == len(high) or len(low) == len(high)+1
```

Then the lower median is `max(low)`.

Example insertion sequence:

```
insert 5: low=[5], high=[]      median=5
insert 1: low=[1], high=[5]     median=1
insert 9: low=[5,1], high=[9]   median=5
insert 2: low=[2,1], high=[5,9] median=2
```

After every insertion:

1. Put the value in the correct side.
2. Swap tops if order is violated.
3. Rebalance sizes.

3 Sweep with ending times

To count rooms for intervals, process intervals by start time. A min-heap stores end times of currently occupied rooms. Before a new interval starts, pop all rooms whose end time is at most the start. The heap size is the number of simultaneous intervals.

For half-open intervals $[l, r)$, an interval ending at time t does not overlap an interval starting at t . Therefore pop while:

```
end_time <= next_start
```

For closed intervals $[l, r]$, the condition would be different.

4 K-way merging

If arrays A and B are sorted, each fixed i gives a sorted stream:

```
A[i]+B[0], A[i]+B[1], A[i]+B[2], ...
```

Put the first item of every stream in a heap. Repeatedly pop the smallest sum and push the next sum from the same stream. This is the same idea as merging many sorted lists.

Worked example:

```
A = [1, 4]
B = [2, 3, 10]
```

Streams:

```
i=0: 3, 4, 11
i=1: 6, 7, 14
```

Start heap:

```
(3,0,0), (6,1,0)
```

Pop (3,0,0), then push (4,0,1). This produces sorted pair sums without generating every pair in advance.

5 Python and C++ heap details

Python's `heapq` is a min-heap. For a max-heap, store negative values:

```
import heapq
low = []
heapq.heappush(low, -x)
largest = -low[0]
```

C++ `priority_queue<int>` is a max-heap by default. For a min-heap:

```
priority_queue<int, vector<int>, greater<int>> pq;
```

For pair sums, store all fields needed to generate the next candidate:

```
(current_sum, i, j)
```

If you store only `current_sum`, you cannot know which next pair to push.

6 Checklist

Define exactly what one heap entry represents.
Make sure stale entries are removed or ignored.
For two heaps, rebalance after every insertion.
For interval problems, decide whether touching endpoints overlap.

7 Local exercises

- A. Running Median: After every inserted number, print the lower median of the prefix.
- B. Course Rooms: Given half-open intervals $[l, r)$, print the minimum number of rooms needed.
- C. K Smallest Pair Sums: Two arrays are sorted. Print the k smallest values of $a[i] + b[j]$.

8 How to use the checker

The local checker compiles/runs both languages and compares output tokens exactly:

```
CP_TARGET=solution python3 sections/39_heaps_priority_queues/check.py
```

Replace `CP_TARGET=solution` by your own `solve.cpp` or `solve.py` attempts when practicing.